



VINF - čo sa robilo

- stránka na scrapovanie: <https://myanimelist.net/manga.php>
- z každej kategórie (celkovo 18) scrapnúť N najviac relevantných máng
- túto logiku v podstate už mám naimplementovanú len ju musím trochu zmeniť, aby mi to pozbieralo viac údajov. zatiaľ nejaké dáta mám, ale málo (~600 stránok, čo je 46.5MB)
- aké info o mänge nás zaujímajú?
 - volumes,
 - chapters,
 - status,
 - odkedy dokedy vychádza(I),
 - žánre,
 - tématiky,
 - magazíny v ktorých manga vychádza,
 - autori,
 - možno nejaké max 3 hlavné postavy? (toto asi skor nie, max tak ze jednu asi)

Vizuál celej stránky:

príklad poscrapovaných linkov

Názov	Dátum úpravy	
myanimelist.net_manga_2_Berserk.txt	11/10/2024 17:28	https://myanimelist.net/manga/2/Berserk
myanimelist.net_manga_13_One_Piece.txt	11/10/2024 17:28	https://myanimelist.net/manga/23390/Shingeki_no_Kyojin https://myanimelist.net/manga/116778/Chainsaw_Man https://myanimelist.net/manga/13/One_Piece
myanimelist.net_manga_23390_Shingeki_no_Kyojin.txt	11/10/2024 17:28	https://myanimelist.net/manga/121496/Solo_Leveling
myanimelist.net_manga_33327_Tokyo_Ghoul.txt	11/10/2024 17:28	https://myanimelist.net/manga/33327/Tokyo_Ghoul https://myanimelist.net/manga/44347/One_Punch_Man
myanimelist.net_manga_44347_One_Punch_Man.txt	11/10/2024 17:28	https://myanimelist.net/manga/96792/Kimetsu_no_Yaiba https://myanimelist.net/manga/75989/Boku_no_Hero_Academia https://myanimelist.net/manga/113138/Jujutsu_Kaisen https://myanimelist.net/manga/11/Naruto https://myanimelist.net/manga/656/Vagabond
myanimelist.net_manga_116778_Chainsaw_Man.txt	11/10/2024 17:28	
myanimelist.net_manga_121496_Solo_Leveling.txt	11/10/2024 17:28	

- scrapnutých 5639 stránok na základe linkov z minulej konzultácie
- vytvorený parser, ktorý uloží všetko do jedného .csv
- parser v podstate extrahuje pomocou regexov potrebné časti, upraví ich do správnej formy a potom uloží do .csv

	A	B	C	D	E	F	G	H	I	J	K
1	Genres	Serialization	Authors	Volume	Chapter	Status	Published	Title	Published From	Published To	
2	Comedy, Drama, Mystery, Supernatural, Se Young Magazine (Weekly)		CLAMP	19	213	Finished	Feb 24, 2003 to Feb 9, 2011	xxxHOLIC	Feb 24, 2003		Feb-09
3	Action, Adventure, Fantasy, Martial Arts, Shounen Jump (Weekly)		Kishimoto Masashi	72	700	Finished	Sep 21, 1999 to Nov 10, 2014	Naruto	Sep 21, 1999		Nov-10
4	Action, Adventure, Award Winning, Fantasy, Shounen Jump (Weekly)		Kubo Tite	74	705	Finished	Aug 7, 2001 to Aug 22, 2016	Bleach	Aug 7, 2001		Aug-22
6	Adventure, Comedy, Fantasy, Shounen Shounen Magazine (Weekly)		Mashima Hiro	35	298	Finished	Jul 7, 1999 to Jul 27, 2005	Rave	Jul 7, 1999		Jul-27
7	Action, Adventure, Comedy, Fantasy, Romi Shounen Magazine (Weekly)		Akamatsu Ken	38	355	Finished	Feb 26, 2003 to Mar 14, 2012	Mahou Sensei Negima!	Feb 26, 2003		Mar-14
8	Award Winning, Comedy, Romance, Ecchi, Shounen Magazine (Weekly)		Akamatsu Ken	14	120	Finished	Oct 21, 1998 to Oct 31, 2001	Love Hina	Oct 21, 1998		Oct-31
9	Comedy, Drama, Romance, School, Shoujo LaLa		Tsuda Masami	21	108	Finished	Dec 22, 1995 to May 10, 2005	Kareshi Kanojo no Jijou	Dec 22, 1995		May-10
10	Award Winning, Comedy, Drama, Romance, Ribon Magazine		Obana Miho	10	54	Finished	Jul 2, 1994 to Oct 3, 1998	Kodomo no Omocha	Jul 2, 1994		Oct-03
11	Action, Comedy, Drama, Mystery, Supernat Shounen Magazine (Weekly)		Kibayashi Shin, Ayamine	39	344	Finished	Mar 24, 1999 to Feb 21, 2007	GetBackers	Mar 24, 1999		Feb-21
12	Award Winning, Drama, Mystery, Adult Cas Big Comic Original		Urasawa Naoki	18	162	Finished	Dec 5, 1994 to Dec 20, 2001	Monster	Dec 5, 1994		Dec-20
13	Award Winning, Comedy, Drama, Supernat Shounen Jump (Weekly)		Obata Takeshi, Hotta Yu	23	198	Finished	Dec 8, 1998 to Jul 14, 2003	Hikaru no Go	Dec 8, 1998		Jul-14
14	Supernatural, Suspense, Psychological, Sh Shounen Jump (Weekly)		Obata Takeshi, Ohba Ts	12	108	Finished	Dec 1, 2003 to May 15, 2006	Death Note	Dec 1, 2003		May-15
15	Action, Drama, Historical, Samurai, Shoun Shounen Jump (Weekly)		Watsuki Nobuhiro	28	259	Finished	Apr 12, 1994 to Sep 21, 1999	Rurouni Kenshin: Meiji Kei	Apr 12, 1994		Sep-21
16	Action, Comedy, Romance, Ecchi, Harem, Shounen Sunday		Takahashi Rumiko	38	407	Finished	Aug 5, 1987 to Feb 21, 1996	Ranma 1/2	Aug 5, 1987		Feb-21
18	Action, Adventure, Award Winning, Drama, Shounen Gangan		Arakawa Hiromu	27	116	Finished	Jul 12, 2001 to Sep 11, 2010	Fullmetal Alchemist	Jul 12, 2001		Sep-11
22	Drama, Romance, School, Visual Arts, Josei Zipper		Yazawa Ai	5	48	Finished	Mar 23, 1999 to Mar 22, 2003	Paradise Kiss	Mar 23, 1999		Mar-22
24	Comedy, Drama, Romance, Crossdressing LaLa		Hatori Bisco	18	87	Finished	Jul 24, 2002 to Sep 24, 2010	Ouran Koukou Host Club	Jul 24, 2002		Sep-24
25	Award Winning, Comedy, Drama, Romance, Bessatsu Margaret		Nakahara Aya	17	68	Finished	Aug 13, 2001 to Aug 13, 2007	Lovely...Complex	Aug 13, 2001		Aug-13
26	Action, Adventure, Comedy, Drama, Fantai Shounen Gangan		Kishimoto Seishi	19	78	Finished	Aug 11, 2001 to Dec 12, 2007	666 Satan	Aug 11, 2001		Dec-12
27	Action, Drama, Romance, Supernatural, M Ribon Magazine		Tanemura Arina	7	33	Finished	Dec 29, 1997 to Jun 3, 2000	Kamikaze Kaitou Jeanne	Dec 29, 1997		Jun-03
28	Comedy, Drama, Romance, School, Shoujo Ribon Magazine		Tanemura Arina	11	59	Finished	Aug 3, 2004 to Jun 3, 2008	Shinshi Doumei Cross	Aug 3, 2004		Jun-03
29	Adventure, Fantasy, Shounen		Mukai Natsumi	10	59	Finished	Dec 27, 1999 to Jan 27, 2005	#NAME?	Dec 27, 1999		Jan-27
30	Action, Adventure, Supernatural, Shounen Shounen Jump (Weekly)		Kubo Tite	4	30	Finished	Jul 20, 1999 to Feb 15, 2000	Zombie Powder.	Jul 20, 1999		Feb-15

- vytvorený indexer nad testovacou množinou dát (cca 50 riadkov v .csv), pre ľahší debug
- indexer vytvorený v elasticsearch - dal sa ľahšie rozbehnúť ako pylucene

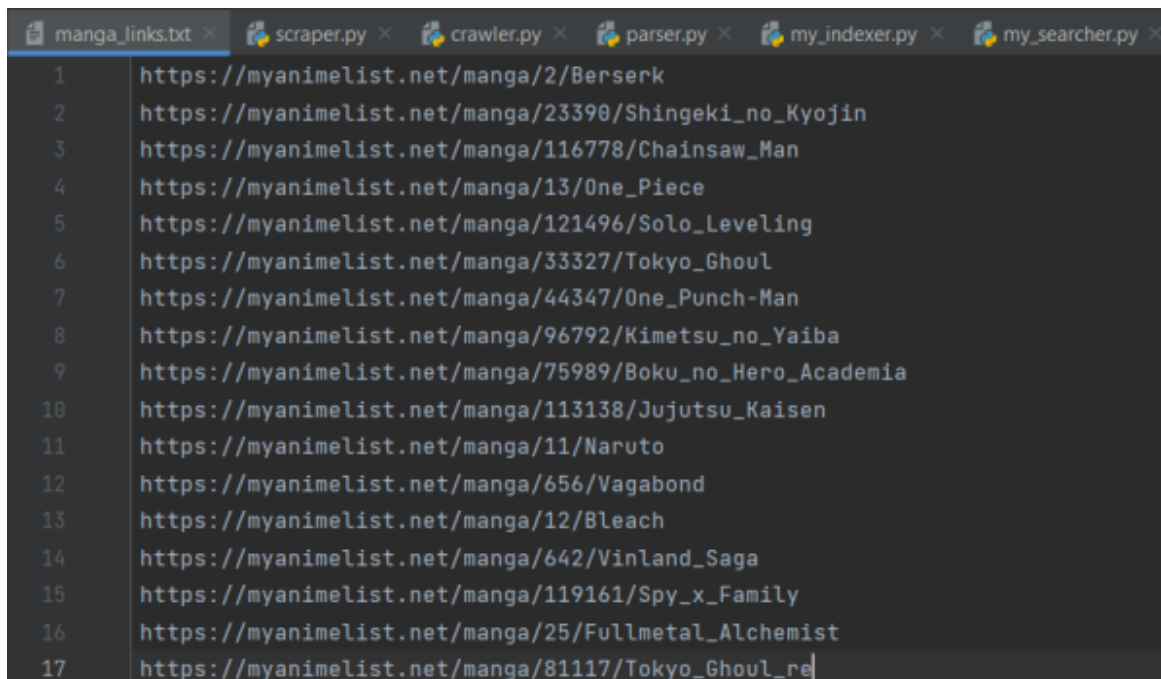
```
document = {
  'title': row['Title'],
  'genre': row['Genres'],
  'authors': row['Authors'],
  'published_date_from': published_from,
  'published_date_to': published_to,
  'serialization': row['Serialization'],
  'volumes': row['Volumes'],
  'chapters': row['Chapters'],
  'status': row['Status'],
}
```

- vytvorený searcher, ktorý väčšinu atribútov filtruje správne (todo: dátum, žánre, možno authors):

```
What do you want to filter?
[1] Author
[2] Genre
[3] Number of volumes
[4] Number of chapters
[5] Status (Finished, On Hiatus, Publishing)
[6] Title
5
Status:Finished
Number of records found: 37
{'title': 'xxxHOLiC', 'genre': 'Comedy, Drama, Mystery, Supernatural, Seinen', 'authors': 'Gengoh Watanabe', 'published_date_from': '2003-05-01', 'published_date_to': '2005-05-01', 'serialization': 'Manga', 'volumes': 14, 'chapters': 14, 'status': 'Finished'}
{'title': 'Naruto', 'genre': 'Action, Adventure, Fantasy, Martial Arts, Shounen', 'authors': 'Masashi Kishimoto', 'published_date_from': '1999-09-01', 'published_date_to': '2014-11-01', 'serialization': 'Manga', 'volumes': 72, 'chapters': 72, 'status': 'Finished'}
{'title': 'Bleach', 'genre': 'Action, Adventure, Award Winning, Fantasy, Shounen', 'authors': 'Tite Kubo', 'published_date_from': '2001-07-01', 'published_date_to': '2012-07-01', 'serialization': 'Manga', 'volumes': 34, 'chapters': 34, 'status': 'Finished'}
{'title': 'Rave', 'genre': 'Adventure, Comedy, Fantasy, Shounen', 'authors': 'Hiroshi Seko', 'published_date_from': '2003-07-01', 'published_date_to': '2012-07-01', 'serialization': 'Manga', 'volumes': 14, 'chapters': 14, 'status': 'Finished'}
{'title': 'Mahou Sensei Negima!', 'genre': 'Action, Adventure, Comedy, Fantasy, Shounen', 'authors': 'Ken Akamatsu', 'published_date_from': '2003-07-01', 'published_date_to': '2012-07-01', 'serialization': 'Manga', 'volumes': 14, 'chapters': 14, 'status': 'Finished'}
{'title': 'Love Hina', 'genre': 'Award Winning, Comedy, Romance, Ecchi, Harlem', 'authors': 'Yasuharu Takeda', 'published_date_from': '1999-07-01', 'published_date_to': '2012-07-01', 'serialization': 'Manga', 'volumes': 14, 'chapters': 14, 'status': 'Finished'}
{'title': 'Kareshi Kanojo no Jijou', 'genre': 'Comedy, Drama, Romance, School', 'authors': 'Yasuharu Takeda', 'published_date_from': '1999-07-01', 'published_date_to': '2012-07-01', 'serialization': 'Manga', 'volumes': 14, 'chapters': 14, 'status': 'Finished'}
```

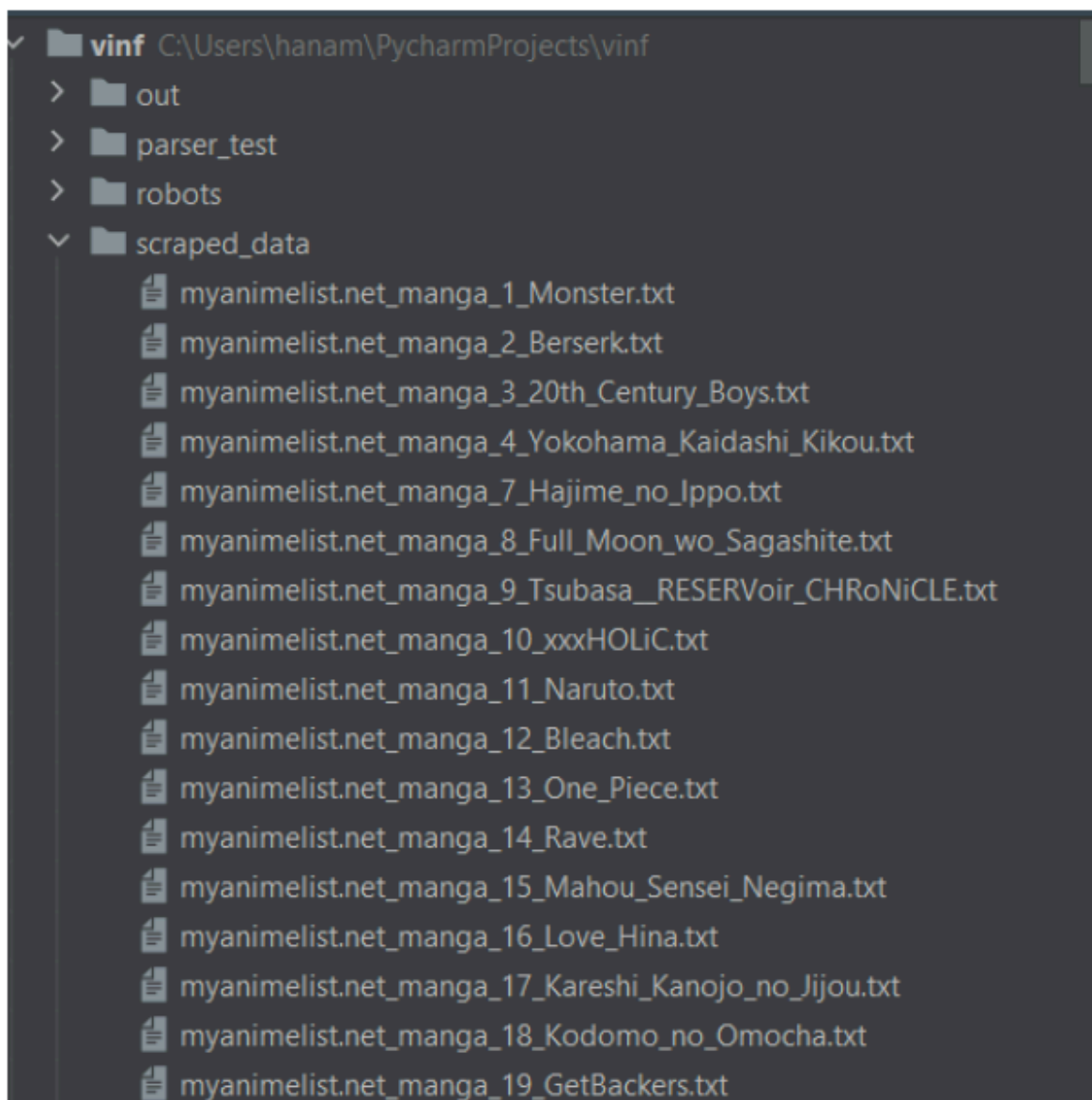
Celá doterajšia časť projektu pozostáva z nasledovných programov:

1. crawler.py - hlavnou úlohou je prechádzať nami zvolenú zdrojovú stránku a vrátiť .txt súbor s linkami, ktoré budeme v ďalšom kroku scrapovať.



```
manga_links.txt x scraper.py x crawler.py x parser.py x my_indexer.py x my_searcher.py x
1 https://myanimelist.net/manga/2/Berserk
2 https://myanimelist.net/manga/23390/Shingeki_no_Kyojin
3 https://myanimelist.net/manga/116778/Chainsaw_Man
4 https://myanimelist.net/manga/13/One_Piece
5 https://myanimelist.net/manga/121496/Solo_Leveling
6 https://myanimelist.net/manga/33327/Tokyo_Ghoul
7 https://myanimelist.net/manga/44347/One_Punch-Man
8 https://myanimelist.net/manga/96792/Kimetsu_no_Yaiba
9 https://myanimelist.net/manga/75989/Boku_no_Hero_Academia
10 https://myanimelist.net/manga/113138/Jujutsu_Kaisen
11 https://myanimelist.net/manga/11/Naruto
12 https://myanimelist.net/manga/656/Vagabond
13 https://myanimelist.net/manga/12/Bleach
14 https://myanimelist.net/manga/642/Vinland_Saga
15 https://myanimelist.net/manga/119161/Spy_x_Family
16 https://myanimelist.net/manga/25/Fullmetal_Alchemist
17 https://myanimelist.net/manga/81117/Tokyo_Ghoul_re
```

2. scraper.py - priamo nadväzuje na crawler. Pracuje s .txt súborom, odkiaľ si berie postupne linky, a scrapuje html obsah každého z nich do nastaveného folderu. Výstupom tohto programu je folder naplnený .txt súbormi - každý súbor obsahuje html kód danej stránky.



3. parser.py - slúži na extrakciu určitých dát pomocou regexov zo zozbieraného html. Okrem extrakcie rieši špecifickejšie niektoré stĺpce ktoré potrebovali trochu viac pozornosti a úprav (serialization, authors, published) Program beží v loope, kde prechádza celým folderom z predchádzajúceho kroku. Výstupom je .csv súbor "manga_results.csv".

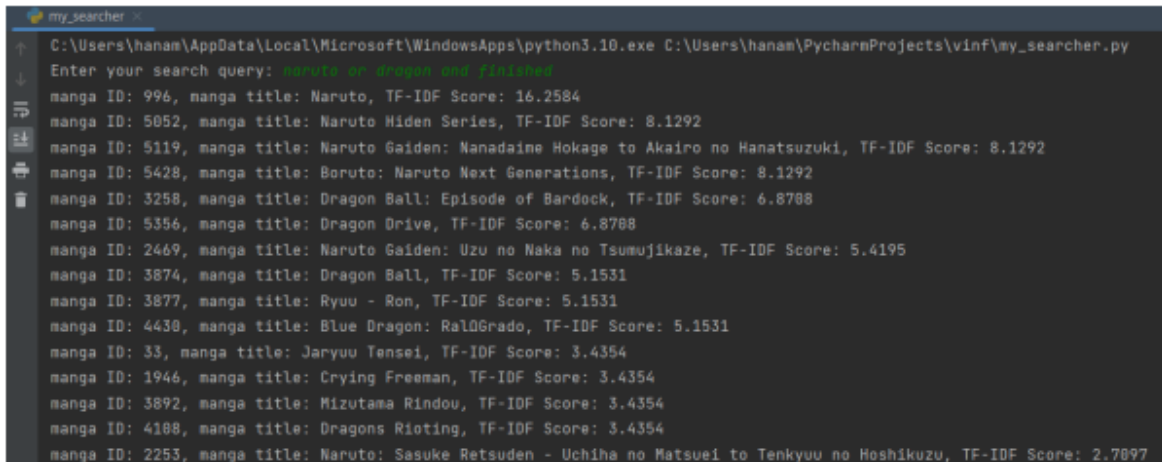
#	A	B	C	D	E	F	G	H	I	J	K	
1	ID	Genres	Serialization	Authors	Volume	Chapter	Status	Title	Synopsis	Published From	Published To	
2	1	Drama, Mystery, It Not found		Nomura, Mizuki; Takei	8	74	Finished	Aqut; Bungaku Shoujo; Kuro	Koroha Inoue lives out his days as a normal high school	Apr 28, 2006	Aug 30, 2008	
3	2	Fantasy, Seinen	Morning Two	Shirahama, Kamome	Unknown	Unknown	Publishing	Tongari Boushi no Atelier	Coco, a humble dressmaker's daughter, has a	Jul 22, 2016	?	
4	3	Comedy, Supernatural	Dengeki Mash	Azuma, Sawayoshi	Unknown	Unknown	Publishing	Onaka na Tereshi wa Akuma	Akutsu Masatora is a demon who has infiltrated a high s	Jun 27, 2016	?	
5	4	Award Winning, Co Not found		Azuma, Hideo	1	3	Finished	Shisou Nikki	Hideo Azuma was born 6th February 1950 in Hokkaido.	Mar-05	Unknown	
6	5	Action, Comedy, Shounen Jump (V	Tamara, Ryoushei		28	250	Finished	Beelzebub	Ishiyama High—land of the delinquents. No man	Feb 23, 2009	Mar 13, 2015	
7	6	Fantasy, Children's	Comic Walker	Hota, CHROLU	Unknown	Unknown	Publishing	Uchi no Ko no Tame naraba,	While on a job deep in the forest, Dale—a young,	Jul 22, 2016	?	
8	7	Award Winning, My Shounen Jump	(V Demizu, Potuaka; Shiro		20	181	Finished	Yakusoku no Neverland	At Grace Field House, life could not be better for the or	Aug 1, 2016	Jun 15, 2020	
9	8	Comedy, Sports, Shounen Jump	Kagami, Yuuma		5	65	Finished	Yuizaki-san wa Nagenai!	Oomiyu-kun is often late for class so as a punishment,	Jul 7, 2016	Dec 21, 2017	
10	9	Action, Sci-Fi	Comic Beam	Usune, Masatoshi	22	139	Finished	Sunabozu	In the future, Japan is a wasteland. In the Great Kantou	Jan 11, 1997	Oct 12, 2020	
11	10	Action, Sci-Fi, Super Shounen Champ	Yamaguchi, Takayuki		11	100	Finished	Kakugo no Susune	Not found	Feb 24, 1994	Mar 28, 1996	
12	11	Action, Fantasy, Shounen Jump	Nakayama, Atsushi		6	51	Finished	Uratarou	As a child, Princess Taira Chiyo escaped from being cap	Jul 28, 2016	Sep 28, 2017	
13	12	Comedy, Fantasy, I Tonari no Young	Izuyouya		9	220	Finished	Musuko ga Kawaiiute Shika!	Not found	May 25, 2016	Jul 22, 2020	
14	13	Boys Love	RuTite	Kazuki, Rai	1	8	Finished	Hatsukoi	Yusuo Hirano always believed he was special and desti	Jul 22, 2016	Jul 22, 2017	
15	14	Boys Love, Drama, Cheri	Beriko, Scarlet		5	29	Finished	Jealousy	Uchi Ragi is in a bind, both literally and figuratively, wh	Jul 30, 2016	May 28, 2021	
16	15	Comedy, Ecchi, Sci Young Magazine	Seborou	Unknown	Unknown	Unknown	Publishing	Nande Koko ni Sensei ga!	Not found	Oct 10, 2015	?	
17	16	Mystery, Suspense, Jump SQ	Takesuchi, Ryousuke; H	Unknown	Unknown	Unknown	Publishing	Yusoku no Moriarty	In the late 19th century, the British Empire has become	Aug 4, 2016	?	
18	17	Gourmet, Slice of Life, Young Gangan	Yamashita, Noriyuki; T		12	121	Finished	Masoki no Pon	Having lost his confidence as a breadmaker, Hoshino M	Aug 21, 2015	Aug 6, 2021	
19	18	Adventure, Fantasy, Comic Ride	Suemitsu, Dicca; Ryuu	Unknown	Unknown	Unknown	Publishing	Kenja no Deshi wo Nanoru Ki	In one of the worlds that exist within virtual reality,	Kagai Jul 28, 2016	?	
20	19	Boys Love, Slice of Life	Hirohara, Meguru		1	6	Finished	Secret XXX	As his father is a zookeeper and his mother a veterinar	Jan 14, 2016	Mar 14, 2017	
21	20	Romance, Sports, I Sho-Comi	Shiwaishi, Yuki		3	17	Finished	Kim wa, Ore ga Suki da te	A brilliant and good at sport prince is mourning because	Apr 5, 2016	Oct 20, 2016	
22	21	Drama, Shounen	Shounen Jump	Taguchi, Shoushi; Mia		3	18	Finished	Jumyou wo Katotte Moratta.	Helpless and struggling for cash, 20-year-old Kusunoki	Aug 10, 2016	Oct 25, 2017
23	22	Slice of Life, Sports	Manga Time Kirai Nyo!	Azai		10	62	Finished	Harukana Receive	The story takes place in Otinawa, where Haruka forms	Aug 24, 2015	Sep 24, 2020
24	23	Gourmet, Romance, Comic Ryu	Ononaka, Akifuro		6	39	Finished	Kumika no Mikaku	In an Earth that has opened its borders to a variety of ali	Aug 19, 2015	Jul 20, 2018	

4. my_indexer.py - ručne implementovaný indexer cez .csv file. v podstate všetok text z "manga_results.csv" je predspracovaný (tokenizácia, odstránenie stopwords, lematizácia - pre náš prípad mi prišla lepšia ako stematizácia). Zároveň je pre každý term vyrátané TF (ráta sa počas tvorby indexu), IDF a TFIDF (rátajú sa po vytvorení celého indexu). Výsledkom je .csv súbor "inverted_index.csv"

	A	B	C	D	E	F	G	H
1	Term	DocIDs	TF	IDF	TF-IDF			
2	aa	3945, 4507	1, 1	3.4501	3.4501, 3.4501			
3	aaah		4087	1	3.7511	3.7511		
4	aaeru		3456	1	3.7511	3.7511		
5	aaltonen		3909	1	3.7511	3.7511		
6	aaron		2140	3	3.7511	11.2534		
7	aback	861, 3271, 3375, 4792, 49	1, 1, 1, 1, 1	3.0522	3.0522, 3.0522, 3.0522, 3.0522, 3.0522			
8	abandon	320, 1143, 1443, 1839, 2911	1, 1, 1, 1, 1, 1, 1	2.7969	2.7969, 2.7969, 2.7969, 2.7969, 2.7969, 2.7969, 2.7969			
9	abandoner	297, 339, 384, 424, 670, 6	1, 1, 1, 2, 1, 1, 1, 1, 1	1.973	1.9730, 1.9730, 1.9730, 3.9459, 1.9730, 1.9730,			
10	abandonin	248, 1762, 5120	1, 1, 1	3.274	3.2740, 3.2740, 3.2740			
11	abara	1800, 3364, 5015	2, 1, 1	3.274	6.5480, 3.2740, 3.2740			
12	abarenbo		5409	2	3.7511	7.5023		
13	abd		5566	1	3.7511	3.7511		
14	abdomen	3976, 4783	1, 1	3.4501	3.4501, 3.4501			
15	abducted	589, 1051, 1113, 2265, 48	1, 1, 1, 1, 1, 1	2.973	2.9730, 2.9730, 2.9730, 2.9730, 2.9730, 2.9730			
16	abductor		2487	1	3.7511	3.7511		
17	abe	1320, 2467, 2755, 2779, 2	1, 1, 1, 1, 1, 1, 2, 1, 1	2.4501	2.4501, 2.4501, 2.4501, 2.4501, 2.4501, 2.4501,			
18	abekawa		5079	1	3.7511	3.7511		
19	abel		993	1	3.7511	3.7511		
20	abella		2259	3	3.7511	11.2534		
21	abeno	1067, 3040, 4558	1, 1, 3	3.274	3.2740, 3.2740, 9.8220			
22	abhorred		1619	1	3.7511	3.7511		
23	abi	4597, 5079	1, 1	3.4501	3.4501, 3.4501			
24	abiko		5385	1	3.7511	3.7511		
25	abilites		1969	1	3.7511	3.7511		

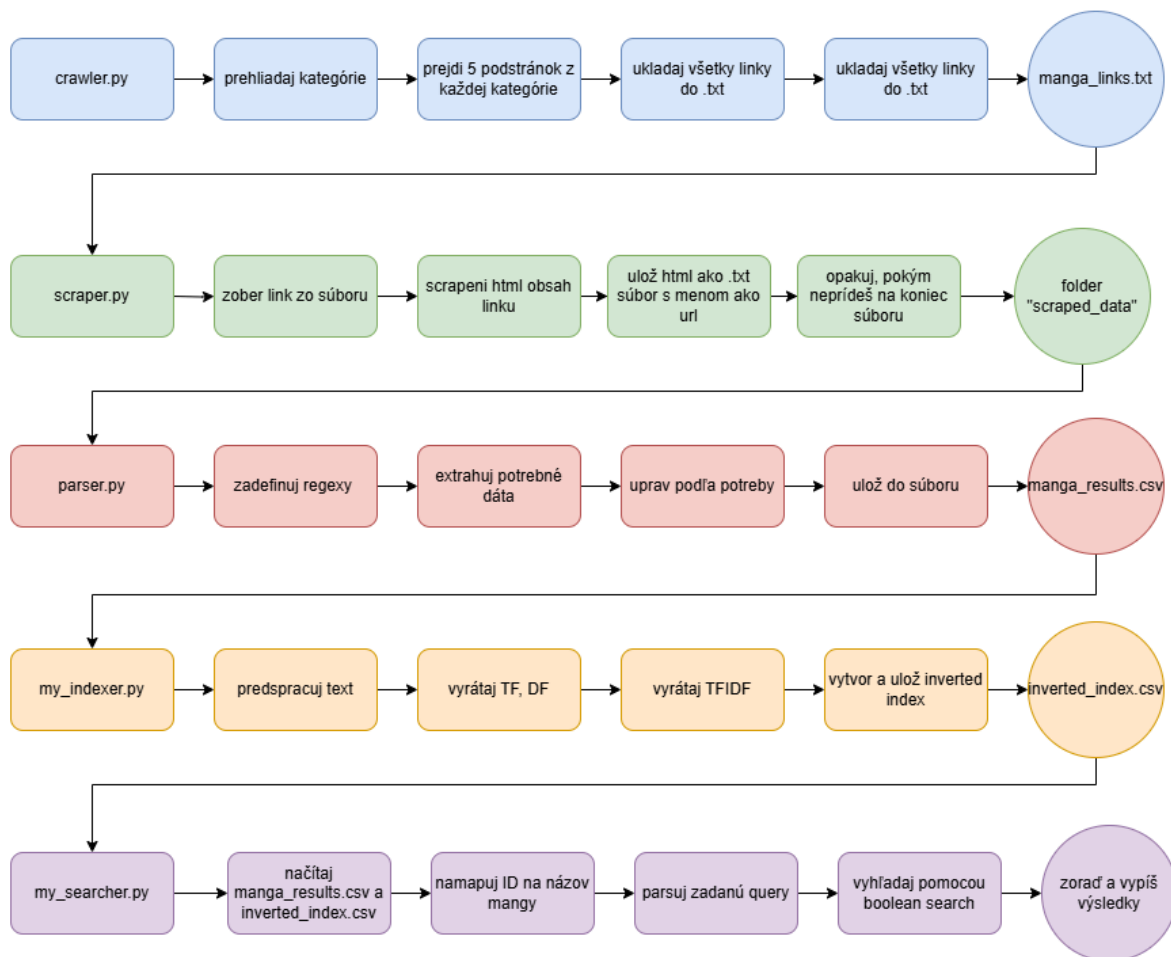
5. my_searcher.py - nadväzuje na indexer a pracuje s vytvoreným "inverted_index.csv". Je možné filtrovať pomocou AND alebo OR a funguje

správne aj pre zložitejšie queries (kombinácia AND a OR operátorov). Filtruje na základe zadanej query - túto zadáva používateľ do terminálu. Následne sa vypíše zoznam všetkých máng spolu s TFIDF skóre, ktoré query vyhovujú. Výsledky sú zoradené podľa TFIDF desc. Na konci programu je v komentári aj príklad niektorých queries.



```
my_searcher
C:\Users\hanam\AppData\Local\Microsoft\WindowsApps\python3.10.exe C:\Users\hanam\PycharmProjects\vinf\my_searcher.py
Enter your search query: naruto or dragon and finished
manga ID: 996, manga title: Naruto, TF-IDF Score: 16.2584
manga ID: 5052, manga title: Naruto Hiden Series, TF-IDF Score: 8.1292
manga ID: 5119, manga title: Naruto Gaiden: Nanadaine Hokage to Akairo no Hanatsuzuki, TF-IDF Score: 8.1292
manga ID: 5428, manga title: Boruto: Naruto Next Generations, TF-IDF Score: 8.1292
manga ID: 3258, manga title: Dragon Ball: Episode of Bardock, TF-IDF Score: 6.8788
manga ID: 5356, manga title: Dragon Drive, TF-IDF Score: 6.8788
manga ID: 2469, manga title: Naruto Gaiden: Uzu no Naka no Tsumujikaze, TF-IDF Score: 5.4195
manga ID: 3874, manga title: Dragon Ball, TF-IDF Score: 5.1531
manga ID: 3877, manga title: Ryu - Ron, TF-IDF Score: 5.1531
manga ID: 4430, manga title: Blue Dragon: Ra10Grado, TF-IDF Score: 5.1531
manga ID: 33, manga title: Jaryuu Tensei, TF-IDF Score: 3.4354
manga ID: 1946, manga title: Crying Freeman, TF-IDF Score: 3.4354
manga ID: 3892, manga title: Mizutama Rindou, TF-IDF Score: 3.4354
manga ID: 4188, manga title: Dragons Rioting, TF-IDF Score: 3.4354
manga ID: 2253, manga title: Naruto: Sasuke Retsuden - Uchiha no Matsuei to Tenkyuu no Hoshikuzu, TF-IDF Score: 2.7897
```

Celý kód momentálne beží na celej vzorke dát - vyše 5k záznamov. Každý program je funkčný aj samostatne, avšak zároveň na seba nadväzujú, ako bolo uvedené v podmienkach. Na diagreame nižšie je bližšie ukázané, ako jednotlivé programy fungujú a ako sú na seba napojené:



- instalacia a rozbehnutie pyspark
- prezretie struktury viacerých wiki dumpov
- stiahnutie 20ks wiki dumpov. ide v podstate o rozkuskovany jeden velky wiki dump (jeden velky dump mi pc nezvlada spracovat)
- definicia regexov na extrakciu dalsich relevantnych informacii (extrahujem navyse este publishers a jednu hlavnu postavu)
- pre obe info bolo potrebne definovat viacero regex vyrazov, kedze to ako boli tieto info reprezentovane v dumpe sa lisilo:

```

25 # regex patterns
26 publisher_regex_1 = r'\\ publisher\\s*=\\s*\\[\\[\\s*(.*)\\s*\\]\\]'
27 publisher_regex_2 = r'\\ publisher\\s*=\\s*\\[\\[\\s*(.*)\\s*\\]\\]'
28 characters_regex_1 = r"\\{\\{Nihongo\\|'\\'+([\\^|]+?)\\'\\'+\\|}"
29 characters_regex_2 = r"(?<=\\{\\{nihongo\\|\\}([\\^|]+))"
30 characters_regex_3 = r"==\\s*Characters\\s*==(?:\\.\\s*\\n)+?;\\s*([\\^\\n]+)"

```

- kod generuje pre kazdy dump jeden csv subor, v ktorom su vyextrahovane nove informacie
- jednotlivé csv spojim spolu do jedného veľkeho csv - príklad na obrázku

	A	B	C	D	E	F
1	Title	publisher1	publisher2	characters1	characters2	characters3
2	Fruits Basket	Hakusensha	Barnes & Noble		Furuba	
3	Dragon Ball (manga)	Shueisha	Viz Media			
4	Love Hina	Kodansha	TV Tokyo		Hina	
5	Fruits Basket	Hakusensha	Barnes & Noble			
6	Beelzebub	Brill (publisher) Brill	University of Chicago Press			
7	Life (manga)	Kodansha			Ayumu Shiiba	
8	Yozakura Quartet	Kodansha	Kodansha		Hime Yarizakura	
9	Toriko	Shueisha	Shueisha	Toriko		
10	Rookies (manga)	Shueisha	Agency for Cultural Affairs			Koichi Kawato
11	One Outs	Shueisha	AnimeNation		Toa Tokuchi	
12	Ahiru no Sora		Kodansha		Sora Kurumatani	
13	B.O.D.Y. (manga)	Shueisha	Anime News Network			
14	Shinobi Life	Akita Shoten	Sharp Point Press			
15	Pride (manga)	Shueisha				
16	Girl Friends (manga)	Futabasha	Sharp Point Press			Mariko Kumakura

- na základe Title spravím left join s mojim pôvodným datasetom (left join, lebo niektoré záznamy z pysparku boli navyše), aby sa dataset obohatil

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	ID	Genres	Serialized	Authors	Volumes	Chapters	Status	Title	Synopsis	Published	Published To	publisher1	publisher2	character	characters	characters3
2	1	Drama, My Not found	Nomura, R		8	74	Finished	Bungaku Shoujo Konoha Inc	Apr 28, 20	Aug 30, 2008						
3	2	Fantasy, Si Morning Tv	Shirahama	Unknown	Unknown		Publishing	Tongari Boushi Coco, a hu	Jul 22, 201 ?							
4	3	Comedy, Si Dengeki M	Azuma, Sa	Unknown	Unknown		Publishing	Oroka na Tenshi Akutsu Ma	Jun 27, 20 ?							
5	4	Award Win Not found	Azuma, Hii		1	3	Finished	Shissou Nikki	Hideo Azur	Mar-05	Unknown					
6	5	Action, Coi Shounen Ji	Tamura, R		28	250	Finished	Beelzebub	Ishiyama F	Feb 23, 20	Mar 13, 2015	Brill (publisher) Brill	University of Chicago Press			

- v mojom pôvodnom datasete je okolo 5600 záznamov, pričom aspoň jeden publisher alebo hlavná postava (vystup z pysparku) bola pridaná ku približne 360 z nich.

Pyspark - spark.py

- prerobila som pyspark - najprv checkujeme na zaklade 3 znakov, ci sa nam stranku oplati spracovavat:
 1. zadefinovala som regexy podla nejakych templates ale i struktury celeho dokument

```
# patterns for initial filtering
manga_infobox = r"\{\{Infobox animanga/Print[^\}]*\}\}"
manga_title = r"\bmanga\b"
manga_section_regex = r"===Manga===
```

2. ak tieto regexy nieco vratia, tak dokument budeme dalej spracovavat

```
# first, we check some key parts here. if these regexes find something, we continue extracting that document
filtered_df = df.filter(
    (f.col("title").isin(name_list_with_disambiguation)) & (
        (f.col("revision.text._VALUE").rlike(manga_infobox)) |
        (f.col("revision.text._VALUE").rlike(manga_title)) |
        (f.col("revision.text._VALUE").rlike(manga_section_regex))
    )
)

# if filtered df is not empty, perform extractions
if filtered_df.count() > 0:
```

3. po checknuti relevancie mozeme dalej spracovavat pomocou dalsieho setu regexov:

```

patterns = data_extraction_patterns

publisher_regex_1 = r'\| publisher\s*=\s*\[\{\s*(.*?)\s*\]\}'
publisher_regex_2 = r'\|publisher\s*=\s*\[\{\s*(.*?)\s*\]\}'
characters_regex_1 = r"\\{\{Nihongo\}\}'+([\^]+?)'\+\\|"
characters_regex_2 = r"^(?<\\{\{nihongo\}\})([\^]+)%"
characters_regex_3 = r"*=\\s*Characters\\s*==(?;.*?\\n)+?;\\s*([\\n]+)%"
aka_regex_1 = r"\\|lead=yes\\}\\}.+?|([\\^|]+)\\+\\|"
aka_regex_2 = r"\\{\\{lit\\}\\}\\s*\"([\\^"])*?\\\"" # lit -> literally
anime_adaptation_regex = r"\\{\\{Infobox animanga/Video([\\^]+)\\}\\}" # if this returns something, then we put "1" in csv file
about_regex = r"*=Plot*=\\n(?;\\{\\{.*?\\}\\}\\n)?(.*?)(?:\\n\\n\\Z)" # takes first paragraph from Plot section
title_regex = r"<title>(.*?)</title>" # title extraction

# first, we check some key parts here. if these regexes find something, we continue extracting that document
filtered_df = df.filter(
    (f.col("title").isin(name_list_with_disambiguation)) & (
        (f.col("revision.text._VALUE").rlike(manga_infobox)) |
        (f.col("revision.text._VALUE").rlike(manga_title)) |
        (f.col("revision.text._VALUE").rlike(manga_section_regex))
    )
)

# if filtered df is not empty, perform extractions
if filtered_df.count() > 0:

```

4. nakoniec si ulozime csv po spracovaní dumpu

Indexer - indexer.py

- implementovaný v elasticsearch
- data sú pred uložením do indexu predspracované (všetko na malé písmena, odstránenie iných ako alfanum. znakov okrem medzier)
- v tomto kroku vytvárame aj jeden veľký concatnutý field všetkých predchádzajúcich fields pre daný riadok, kvôli hľadaniu podľa fráz
- výstupom programu je hotový index
- na obrázku je kód ktorý index vytvára

```

# index creation
if not es.indices.exists(index=index_name):
    es.indices.create(index=index_name, body={
        "settings": {
            "number_of_shards": 1,
            "number_of_replicas": 0
        },
        "mappings": {
            "properties": {
                "title": {"type": "text"},
                "content": {"type": "text"},
                "genre": {"type": "text"},
                "author": {"type": "text"},
                "published_date_from": {"type": "date"},
                "published_date_to": {"type": "date"},
                "serialization": {"type": "text"},
                "volumes": {"type": "text"},
                "chapters": {"type": "text"},
                "status": {"type": "text"},
                "synopsis": {"type": "text"}
            }
        }
    })
    print(f"Index '{index_name}' created successfully.")

```

Searcher - searcher.py

- casti search query sa predspracuju tak ako pri indexovani
- vyhladavat sa vedia:
- frazy
- jednoduchy search bez logickych clenov (genre: comedy)
- search s logickymi clenmi AND, OR, NOT a ich kombinacie (genre: comedy AND NOT title: city)

- niektoré zložitejšie queries boli ručne skontrolované na testovacej množine a vracali správne výsledky
- na obrázku je príklad spracovania query ktorá obsahuje AND

```
elif 'AND' in user_input:
    conditions = [part.strip() for part in user_input.split('AND')]
    must_clauses = []
    for condition in conditions:
        field, value = parse_condition(condition)
        must_clauses.append({"match": {field: value}})
    query = {
        "query": {
            "bool": {
                "must": must_clauses
            }
        }
    }
```